



Universae

MESERO APP

Desarrollo de Aplicaciones Multiplataforma
Curso 2025/2026

Presentado por:
Roberto de Frutos
Jiménez

Índice

Índice.....	1
1. Resumen	2
2. Introducción	3
3. Relación del proyecto con las asignaturas cursadas.....	4
4. Metodología y planificación.....	5
Objetivos del proyecto	7
Objetivos generales:.....	7
Objetivos específicos:	7
Alcance del proyecto	7
Tecnologías utilizadas	7
Lenguaje de programación.....	8
Entorno de desarrollo	8
Arquitectura y patrones	8
Interfaz de usuario	8
Persistencia de datos.....	8
Comunicación	8
Prototipado y control de versiones.....	8
Evidencias de uso de tecnologías.....	9
Análisis del sistema.....	11
Implementación.....	11
Manual de usuario.....	11

1. Resumen

MESERO APP es una herramienta móvil construida para Android, elaborada en Java con Android Studio, orientada a facilitar la gestión de bares y de sus comandas de forma sincrónica. La aplicación permite gestionar mesas, productos y pedidos, controlar stock, emitir facturas electrónicas y recibir notificaciones sobre pedidos listos. La persistencia de los datos se hace por medio de Room y SharedPreferences, las listas se actualizan automáticamente porque se soportan por LiveData. La arquitectura del sistema es modular y se basa en MVVM, lo que permite separar las responsabilidades y escalar el proyecto. Durante el desarrollo se realizaron pruebas de caja blanca para validar el funcionamiento correcto de los distintos módulos y de la estabilidad del sistema. El proyecto une conocimiento de frontend y backend, esto es, diseño de interfaz, lógica de negocio y control de datos.

Palabras clave: Android, MVVM, Room, gestión de comandas, aplicación móvil, persistencia de datos, Java

An abstract:

This project creates an application that provides the necessary tools to optimize the management of bars, to help make the order process from the bar to customer more efficient and faster. The functionalities of this application include managing tables, managing products, managing orders with an inventory control, sending bills to customers by email, and receiving notification emails when their orders are completed. The application will persistently store data using Room and SharedPreferences. The application also uses LiveData to allow list to automatically update.

This application has been built in accordance with a modular architecture (based on MVVM) which allows for separation of concerns and increases the ability to expand the project. This project was tested using white box testing to ensure modules work correctly and operate reliably.

This project includes both frontend and backend knowledge, including user interface design, programming logic and data management.

Key Words: Android, MVVM, Room, order management, mobile application, data persistence, Java.

2.Introducción

Mesero APP es una aplicación para teléfonos móviles con sistema operativo Android.

Esta fue desarrollada en JAVA con el entorno de desarrollo Android Studio.

La finalidad de esta aplicación es la gestión de bares y restaurantes de una manera digitalizada. El proyecto es la respuesta al control de mesas, productos y pedidos en tiempo real, así como la reducción del número de errores y mayor agilidad en el servicio. La aplicación permite la gestión de mesas, listado de pedidos (generación de pedidos y gestión), control de stock de productos, envío automático de la factura a través de correo electrónico y la recepción de notificaciones sobre los pedidos listos.

La arquitectura utilizada es MVMM para permitir un desarrollo modular y escalable de la aplicación. La persistencia es conseguida a través de Room y Shared Preferences y la sincronización es gestionada por Live Data.

El proyecto es de código abierto sin ninguna licencia aplicada.

Enlace a él GitHub con el código fuente del proyecto: [MESERO APP](#)



Imagen del logo de la aplicación.

3.Relación del proyecto con las asignaturas cursadas

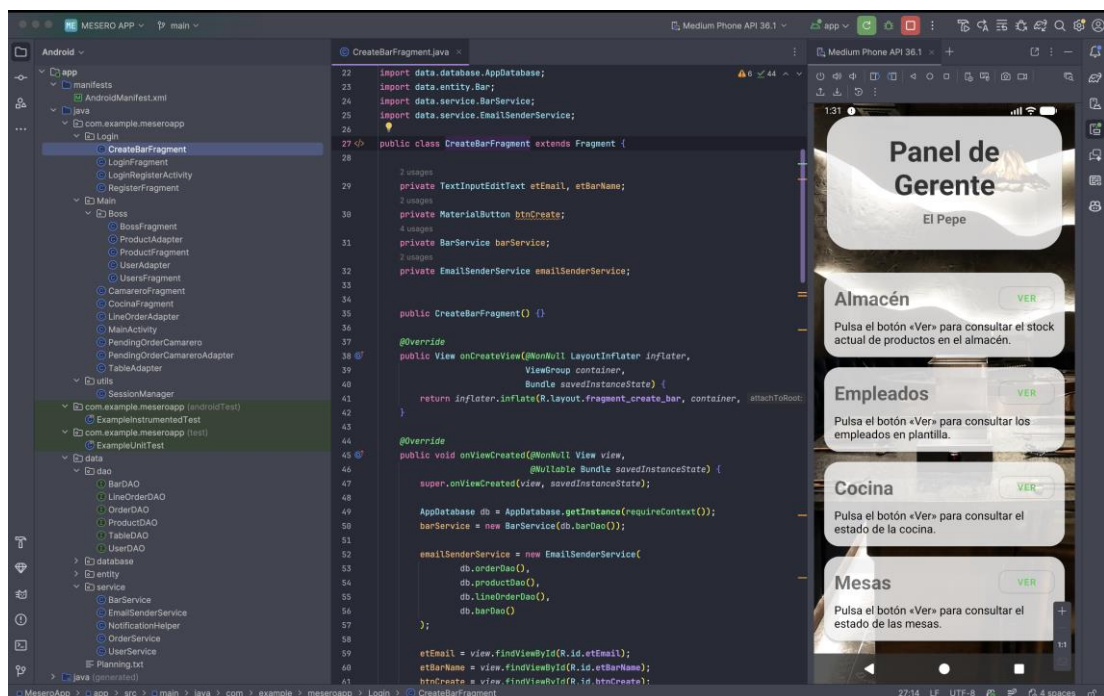
Acceso a datos	Entornos de desarrollo	Bases de datos	Lenguaje de marcas	Programación Multimedia	Empresa
----------------	------------------------	----------------	--------------------	-------------------------	---------

Room, DAO, SQL	Android studio, depuración	Modelo relacional	XML layouts	UI Android	Viabilidad comercial
----------------	----------------------------	-------------------	-------------	------------	----------------------

En esta sección, vamos a desglosar las tecnologías, herramientas y patrones que se han utilizado en el desarrollo de la aplicación MESERO APP, así como su propósito dentro del proyecto.

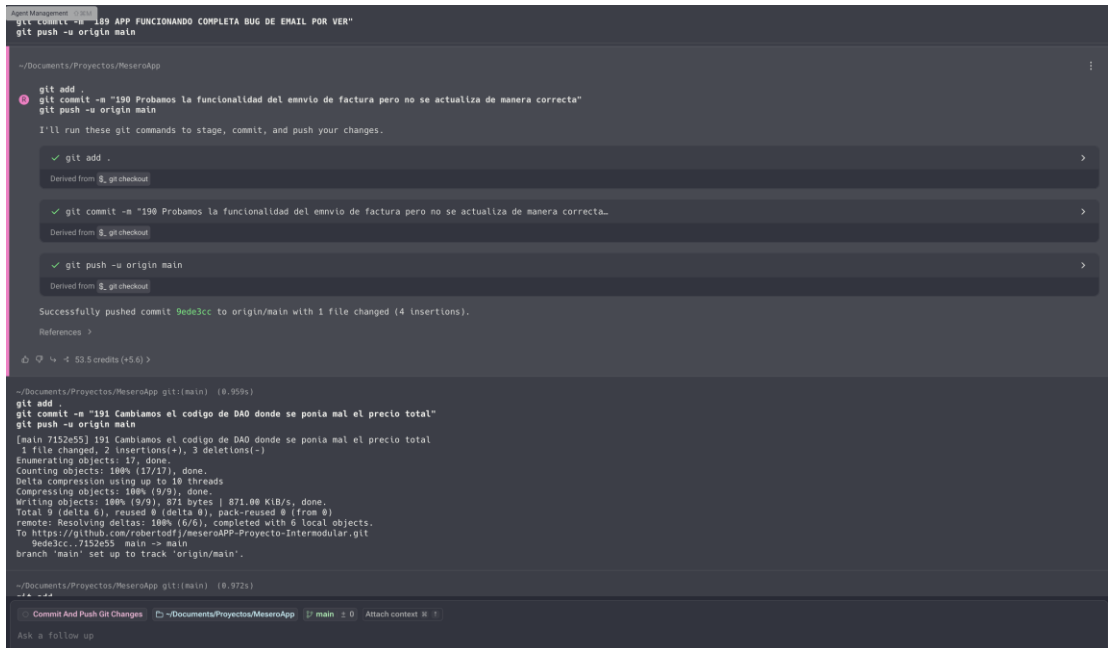
Lenguaje de programación

- Java: Este es el lenguaje principal que hemos utilizado para desarrollar la aplicación en Android.



Entorno de desarrollo

- Android Studio: Es el IDE que hemos elegido para crear, ejecutar y depurar la aplicación.
- Warp: Esta terminal nos ayuda a gestionar comandos y controlar el proyecto.



The screenshot shows the Warp terminal interface with a dark theme. At the top, a prompt indicates the user is in the 'MESERO APP' project. The terminal displays a series of git commands and their output:

```
git commit -m "189 APP FUNCIONANDO COMPLETA BUG DE EMAIL POR VER"
git push -u origin main

~/Documents/Proyectos/MeseroApp
git add .
git commit -m "190 Probamos la funcionalidad del envio de factura pero no se actualiza de manera correcta"
git push -u origin main

I'll run these git commands to stage, commit, and push your changes.

✓ git add .
Derived from 8, git checkout

✓ git commit -m "190 Probamos la funcionalidad del envio de factura pero no se actualiza de manera correcta."
Derived from 8, git checkout

✓ git push -u origin main
Derived from 8, git checkout

Successfully pushed commit 9ede3cc to origin/main with 1 file changed (4 insertions).
References >

🔍 🔖 ⌕ 53.5 credits (+5.6) >
```

Below this, the terminal shows the output of the git push command, including details about the commit, the files changed, and the push progress.

```
git add .
git commit -m "191 Cambiamos el codigo de DAO donde se ponía mal el precio total"
git push -u origin main

[main 7152e55] 191 Cambiamos el codigo de DAO donde se ponía mal el precio total
1 file changed, 2 insertions(+), 3 deletions(-)
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 871 bytes | 871.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com:robertofj/meseroAPP-Proyecto-Intermodular.git
9ede3cc..7152e55 main -> main
branch 'main' set up to track 'origin/main'.
```

At the bottom, the terminal shows the status of the repository and the push progress.

```
~/Documents/Proyectos/MeseroApp git:(main) (9.97s)
Commit And Push Git Changes 📁 ~/Documents/Proyectos/MeseroApp 📌 main 📊 Attach context 🔍 ⌕
Ask a follow up
```

Arquitectura y patrones

- MVVM (Model–View–ViewModel): Este patrón arquitectónico nos permite separar la lógica de negocio de la interfaz de usuario.
- LiveData: Lo utilizamos para actualizar la interfaz en tiempo real de manera automática.

Interfaz de usuario

- RecyclerView: Ideal para mostrar listas dinámicas.
- AlertDialog: Perfecto para mostrar mensajes y obtener confirmaciones.
- Fragments: Nos permiten tener una interfaz modular y reutilizable.

Persistencia de datos

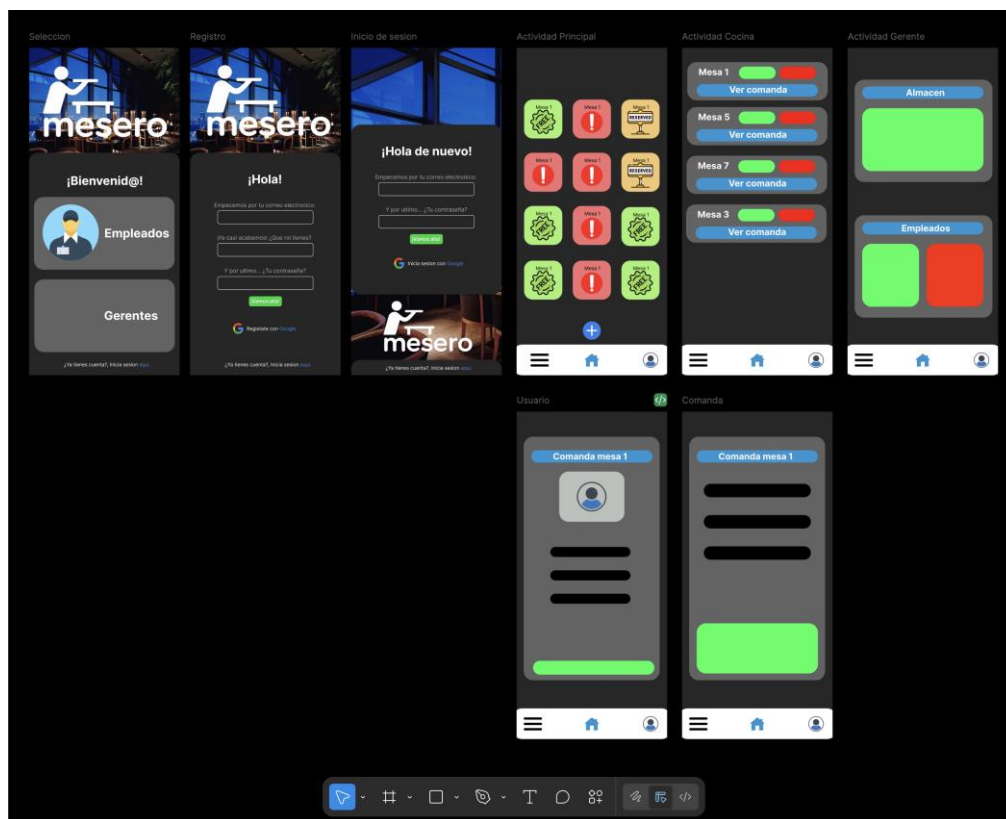
- Room: Base de datos local para almacenar la información de manera estructurada.
- SharedPreferences: Usado para mantener la sesión y las configuraciones básicas.

Comunicación

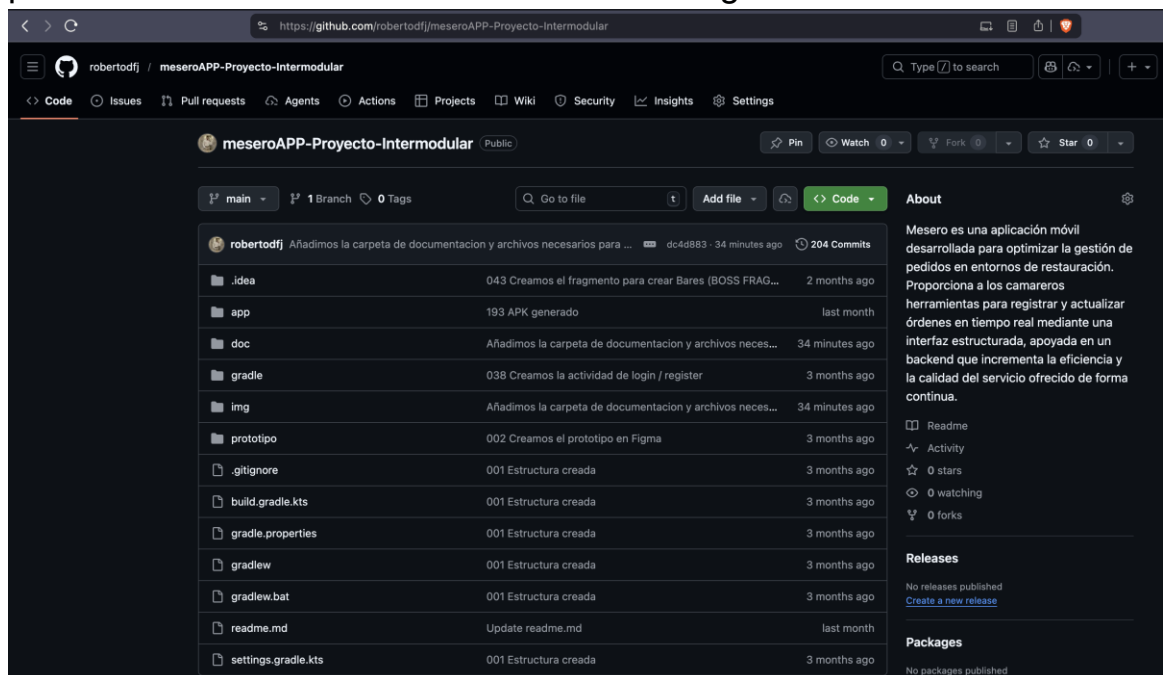
- SMTP: Utilizamos este protocolo para enviar facturas por correo electrónico a través del servicio EmailSenderService.

Prototipado y control de versiones

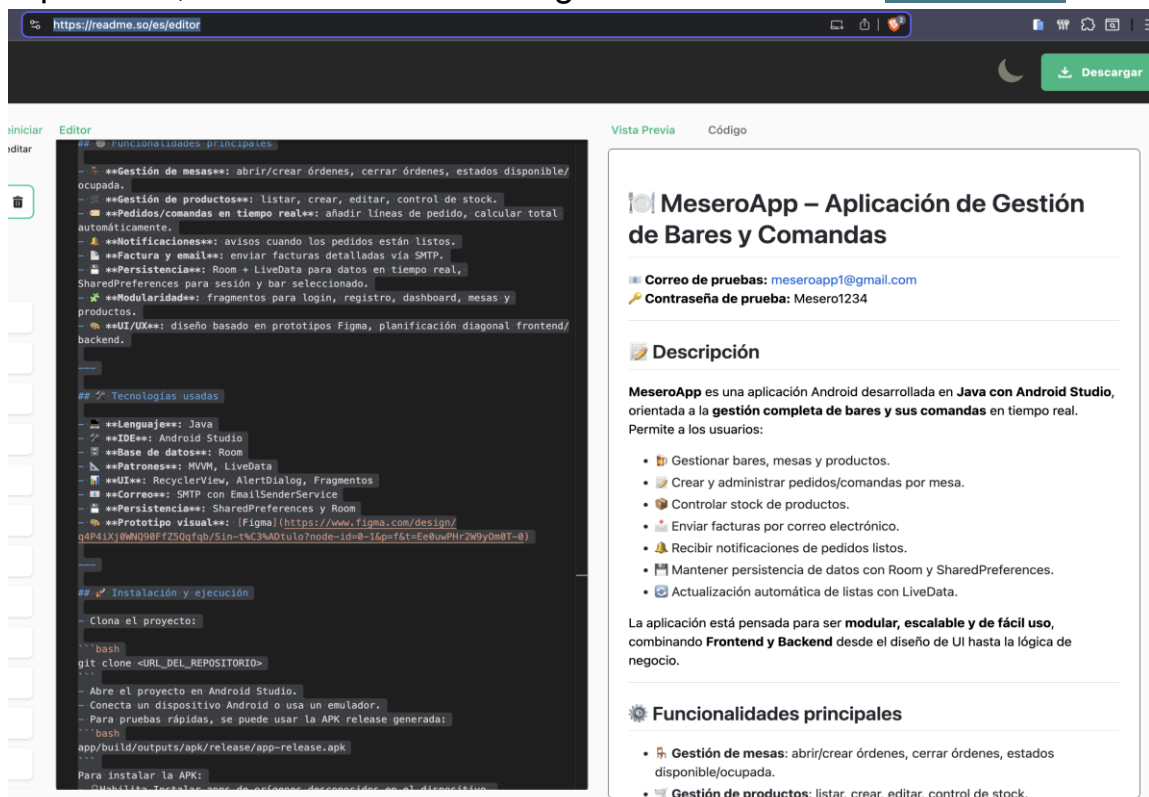
- Figma: Herramienta que utilizamos para crear prototipos visuales de la interfaz.



- GitHub: Nos ayuda a llevar el control de versiones del proyecto y a publicar el código fuente.



- README.md: Este archivo contiene la documentación principal del repositorio, creamos este con la siguiente herramienta readme.so.



4. Metodología y planificación

La aplicación de MESERO APP fue desarrollada con una metodología ágil para un proyecto individual, en esta se combinaba la planificación semanal con revisión de código y avances. Así se consigue iterar sobre funcionalidades, corregir errores y mejorar el diseño para que esta APP cumpla con los objetivos planteados.

Se priorizo la entrega del proyecto en la fecha de entrega de seguimiento de proyecto (8 de marzo) para poder realizar los cambios necesarios aconsejados por el profesorado en la entrega final, así sean sobre funcionalidad, diseño, usabilidad, rendimiento o calidad del código.

Se definieron unas pautas con fases de trabajo para realizar semanalmente desde el planteamiento inicial, desarrollo de módulos, pruebas unitarias y de integración, hasta la documentación final del proyecto. Cada fase incluye unos objetivos claros con actividades a realizar y pruebas para evaluar el proyecto de manera activa.



Imagen sobre planificación obtenida en [HacerTFG](#)

// AÑADIR AQUI LA TABLA GENERADA DE PLANIFICACION

5. Objetivos del proyecto

5.1 Objetivos generales:

Desarrollar una aplicación móvil para dispositivos con sistema operativo Android para la gestión rápida, sencilla y eficaz de pedidos, stock, mesas y comandas en el sector de la hostelería.

5.2 Objetivos específicos:

- Crear un sistema para gestionar mesas y pedidos.
- Desarrollar un control de stock para los productos.
- Implementar la persistencia de datos.
- Enviar facturas por correo electrónico a través de SMTP.
- Configurar notificaciones para alertar sobre pedidos listos desde cocina.
- Diseñar una interfaz amigable creando un prototipo en Figma.

6. Alcance del proyecto

En este apartado se describen las funcionalidades que ofrece MESERO APP:

- Gestión de mesas: abrir/crear órdenes, cerrar órdenes, estados disponible/ocupada.
- Gestión de productos: listar, crear, editar, control de stock.
- Pedidos/comandas en tiempo real: añadir líneas de pedido, calcular total automáticamente.
- Notificaciones: avisos cuando los pedidos están listos.
- Factura y email: enviar facturas detalladas vía SMTP.
- Persistencia: Room + LiveData para datos en tiempo real, SharedPreferences para sesión y bar seleccionado.
- Modularidad: fragmentos para login, registro, dashboard, mesas y productos.
- UI/UX: diseño basado en prototipos Figma, planificación diagonal frontend/backend.

Análisis técnico del sistema

El sistema está estructurado en capas arquitectónicas:

1. Capa de Modelo (Entities): Clases Java que representan entidades (Mesa, Producto, Pedido, DetallePedido, Factura) y sus atributos.
2. Capa DAO: Interfaces DAO que definen métodos de acceso a la base de datos (consultas SQL, inserciones, actualizaciones). Room genera la implementación automática de estos DAOs .
3. ViewModel: Clases ViewModel que exponen datos observables (LiveData) para la UI y manejan la lógica de negocio simple, aislando la interfaz de los detalles de datos.
4. Repository: Clases Repositorio que centralizan la lógica de acceso a datos; el ViewModel usa repositorios para obtener/almacenar información (este es el «único origen de la verdad» para los datos).
5. Fragments (UI): Componentes de interfaz (login, registro, dashboard, listado de mesas, detalle de productos, etc.). Observan LiveData del ViewModel para actualizar dinámicamente la vista.
6. Servicios: EmailSenderService, servicio en segundo plano que envía facturas por correo usando el protocolo SMTP.
7. SessionManager: Componente que maneja la sesión de usuario activa (token de login) y lee/escribe en SharedPreferences la configuración básica.

Cada módulo de código realiza su función específica dentro de la arquitectura: los ViewModel obtienen datos del repositorio, los DAOs interactúan con la base de datos, y los Fragments solo se encargan de mostrar información y responder a la interfaz de usuario.

Conclusiones y proyeccion futura

Conclusiones

Con este proyecto de desarrollo de una aplicacion movil se aprendieron elementos necesarios para mejorar en futuros proyectos, desde la planificacion con tareas especificas, pasando por la tecnica de desarrollo agil, separacion de responsabilidades por capas

- Qué has aprendidoQué dificultades encontraste
- Qué soluciones técnicas aplicaste

Proyección futura

- Sincronización en la nube (Firebase)
- Pasarela de pago
- Versión multiplataforma
- Control multiusuario en red local
- Panel web administrativo

Referencias

[Torunoğlu, Z. \(2024\)](#). MVVM in Android. Medium. MVVM is described as a pattern that “separates the concerns of your application into distinct layers, leading to cleaner, more testable, and maintainable code” .

[Android Developers](#). (n.d.). LiveData overview. Google. “Room generates all the necessary code to update the LiveData object when a database is updated... [keeping] the data displayed in a UI in sync with the data stored in a database” .

[Android Developers](#). (n.d.). Save simple data with SharedPreferences. Google. SharedPreferences “points to a file containing key-value pairs and provides simple methods to read and write them” .

[Figma](#). (n.d.). Free Prototyping Tool: Build Interactive Prototype Designs. Figma. Prototyping tools “make it easy to build and share high-fidelity, no-code, interactive prototypes” .

[Asana](#). (2025). What is Agile methodology? (A beginner’s guide). Asana. Agile methodology “breaks work into small, manageable cycles, commonly known as sprints” and is an “iterative process” .

[GeeksforGeeks](#). (2025). White box Testing - Software Engineering. GeeksforGeeks. “White box testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application” .

[GitHub](#). (2024). What is Version Control?. GitHub. Version control “is essential to the overall health of the software development lifecycle (SDLC) and improves project management and product development” .

[StatCounter Global Stats](#). (2026). Mobile Operating System Market Share Worldwide. StatCounter. In January 2026, Android held 70.36% of global mobile OS market share , highlighting its dominance for mobile app development.